HARDWARE_INVENTORY_MEMO — 硬件与层划分口径核实

对应：`08.08`继续推进手册`.md` §1 V0-2(b)（8/9 到期） 版本：2026-08-08 | 代码版本：主仓库 `059a7fc` **结论一句话**：GPU=`used_outside_A-F`（720 池 CUDA 侧确为 RTX6000/RTX6000D，不在主稿 A–F 的 RTX3090 定义内）；C3=`impl_16-27_is_authoritative`（28 层模型，主稿/手册§1 的"16–23"错误）；模型=`consistent_Qwen3-0.6B`（全部实验 Qwen3-0.6B，主稿全文未点名模型）。

---

0. 时序表（对齐真实 E3-T 端到端）与 V0-2(a) 回答

真实管线（P0E3_TRACE_DELIVERY_HANDBOOK，2026-08-09 三节点一次运行）：Requester 加密请求 → rank0(mini1) 解密+首分片 → activation_exchange → rank1(RTX3090)/rank2(RTX6000) 下游捕获/注入 → scripted post-run 离线验证对账。 **实测分片（本次运行，动态）**：mini1=rank0 层**0–1**、RTX3090=rank1 层**1–4**、RTX6000=rank2 层**4–28**——EXO 按节点 `ram_available` 动态分配，**重跑会变**（历史 memo 的 0–22/22–28 等均不同），不许照抄。

#	真实相位（E3-T walkthrough 14 phase）	实测（E3-T 一次运行）	谁生成 seed	谁持有	执行节点此刻可见什么	代码位置
1	**publish_ciphertext → placement/commit**：Requester 发 verifiable 加密请求，放置绑定 provider	—	**无 seed 生成**（放置只绑定 provider 与加密 payload）	—	placement 元数据 + 密文 payload（下游不读明文）	`verifiable/placement.py:10`；`client.py:19-69`（`build_verifiable_chat_request`）；`crypto.py:40-65`（X25519）；`shared/types/events.py:150`、`shared/apply.py:508`
2	**release_access_package → fetch → decrypt（rank0 ingress, mini1）**：下游只 fetch access 包；rank0 解密输入	rank0 `decrypted_envelope`、`ciphertext_bytes_fetched=176`、`private_input_accessed=True`；下游 `shape_only_dummy`、`0` bytes、`False`	无	—	rank0 明文；下游只拿 upstream activation，**不读明文**（`payload_access_table.csv` 实锤）	`verifiable/runtime.py:136-200`（下游 shape-only、不触密钥）；`E2Edocs/scripts/p0e3_trace_request.py`
3	**执行前（任务已放置，分片待跑）**	—	seed 是**源码常量**：线上 TSTC 默认 `0`、P0–E2 verifier 默认 `selection_seed=0`、SR 主表写死 `2026`、E3 脚本写死 `9527`	**共享源码（所有节点运行同一份）**	**可见 seed 常量 + 派生函数**（`SHA256("accountedge-e2:{seed}:{purpose}")→PCG64` 或 SR 主表 `default_rng(seed)`）——节点从此刻起即可算出将检查的坐标	`tstc/sketch.py:18-21`；`accountedge_e2/verifier.py:27,72-75`；`<SRR>/run_p0_main_table_720.py:37-38`；`E2Edocs/scripts/p0e3_3node_localize.py:110`
4	**shard_exec_C1/C2/C3 + activation_exchange**（三节点跑分片 + 边界交换；B1/B2 hook 捕获/注入）	总推理 **1.28s**；shard_exec C1=887.9ms / C2=890.9ms / C3=886.5ms（trace span 按 rank+时窗过滤）；B1(rank1) 恒等 ×1.0 捕获 ratio 全 1.0000；B2(rank2) 注入 ×2.0 捕获 ratio 全 2.0000（21 对）	无	—	**收到 honest 边界张量 H**（接收本就是执行的一部分）；capture 存 H、inject 生成 H；after-inject 存 H；**全程无 seed**	`worker/engines/mlx/boundary_hook.py:43-66`（capture/inject/after-inject 纯张量）；`attack_injector.py`
5	**challenge_issue**	scripted post-run（离线验证器）	脚本常量 `9527`	离线脚本	**无在线 challenge**——E3-T 手册 §4.4/§7.7 明说 challenge/reveal/verify/localize/settle 全部是 scripted post-run，非在线实时	`p0e3_3node_localize.py:110`
6	**sketch_reveal**	离线	—	—	**无在线 reveal 实现**	（无运行时 reveal 代码）
7	**verify → localize → settle → log_append**：离线 `VerifierChain.evaluate` 对比 H vs H′，first-mismatch 定位归因	**first_mismatch = B2**（21/21 ratio=2.0000，逐对相等）；B1 恒等 0 误报；归因到 rank2/RTX6000/node `2720786b…`	脚本常量	离线脚本	seed 硬编码；坐标由公开派生函数确定	`tstc/verifier.py:55-113`；`accountedge_e2/verifier.py:297+`；`p0e3_3node_localize.py`、`p0e3_trace_collect.py`

要点： 1. **从 publish 到 verify，没有任何协调方持有、且对执行节点保密的 seed。** seed 从第 3 行起就躺在节点运行的共享源码里——真实 E3-T 的执行节点（mini1/RTX3090/RTX6000）跑的就是这份代码。 2. **真实执行节点全程不接触任何"秘密 seed"**：rank0 解密后跑分片、rank1/rank2 接收边界张量后捕获/注入，看到的唯一 seed 是它们自己源码里的常量（`0` / `2026` / `9527`）。 3. **challenge/reveal/verify 在真实管线里不存在**：E3-T 一次运行把它们做成 **scripted post-run 离线验证器**（`p0e3_3node_localize.py`），seed 硬编码 9527；线上 TSTC 模块（`evaluate_chain` / `capture_*`）只被测试调用，未接入运行时。 4. **rank 动态分片不影响 seed 结论**：本次 0–1/1–4/4–28 只是 EXO 按内存动态切的一次实现；无论分片怎么变，节点"知道张量形状（执行时天然知道）+ seed 常量（源码里有）"⇒ 执行前即可精确复现 verifier 将检查的坐标（A1.2：72/72）——这是 `node_sees_before_exec` 的直接时序证据。

V0-2(a) 回答：sketch seed 对执行节点保密吗？

判定：`node_sees_before_exec`（三选一中的 ② 节点执行前即可见）。

三种可能逐一排查： – ① **`node_never_sees_seed`（节点全程不可见）：否。** 该情形要求存在协调方持有的秘密 seed、且永不进入节点代码。当前实现**无 PRF、无协调方密钥、无承诺机制**（`grep PRF/secret/commitment` 三处源码 0 命中），seed 是共享源码里的公共常量——节点从执行前起就能看到。 – ② **`node_sees_before_exec`（节点执行前即可见）：是。** seed 常量（`0` / `2026` / `9527`）在任务放置前就躺在所有节点运行的共享源码里；节点知道张量形状（执行时天然知道）+ seed 常量，即可在执行前算出将检查的坐标。A1.2 实证：72/72 精确复现（见 VERIFIER_SEED_SECRECY_MEMO §2）。 – ③ **`implementation_ambiguous`（未明确区分/取决于配置）：部分成立，但主判定不变。** 跨框架 seed 派生确实未统一（SR 主表 `default_rng` vs verifier `SHA256` 包装，同 seed 坐标不同），但这是"派生方式不一致"，不是"节点看得到看不到"——节点在所有情况下都看得到 seed。

对 §3.3 与新协议增益的影响： – **§3.3 必须写进已知局限**：现有协议"执行前固定 sketch policy/seed"下，节点执行前即可见 seed ⇒ 可在执行前算出抽查坐标并针对构造扰动——这是现有设计的**真实弱点**（不是"把偶然性质结构化"）。 – **新协议增益更大**：正因为现有协议节点知道坐标，commit-then-challenge 的"坐标保密"补的是一个**真洞**。A1.3 定量：sign-balanced 攻击在 KNOWN（seed 已知）下全逃避（TPR≈FPR=0.96%）、SECRET（seed 保密）下 29.4% 检出，**protective_gain≈+0.28**（q=64/256 一致）。

---

1. 设备/层划分/模型总表

stack_id	机器	GPU/芯片	backend	精度	本轮哪些实验用了它	是否在原 A–F 定义内
stack_a_720	**mini1（[已脱敏]）**——已实证（§2.1）	Apple M4 / MPS	torch_mps	BF16（存为 fp32）	P0 主表 §6.4、P1 §6.6、P2 §6.5、W8A16 §6.2	**否**（A–F 无"全前缀 MPS BF16"同构 stack）

stack_id	机器	GPU/芯片	backend	精度	本轮哪些实验用了它	是否在原 A–F 定义内
stack_b_720	**RTX6000**	**NVIDIA RTX 6000D ×2**（Blackwell sm_120）	torch_cuda	FP32	P0 主表 §6.4、P1 §6.6、P2 §6.5、W8A16 §6.2（W8A16 明确写 RTX6000）	**否**（A–F 所有 CUDA stage = RTX3090）
（对照）RTX3090	[已脱敏]	GeForce RTX 3090	torch_cuda	—	40/200 旧池 B/D（run_full28_capture 混合 stage）	是（A–F 的 CUDA 定义）
主稿 A–F	2× Mac mini M4 + RTX3090	M4/MPS + RTX3090/CUDA	torch_mps/cuda	FP16/BF16/FP32/Metal-int8	论文 `tab:eval-execution-stacks` 声明的全部 stack	—（定义本身）

原始证据：`workspace/V0-2/results/b/device_facts.csv`（RTX6000/RTX3090 的 `nvidia-smi -L` + mini1 的 MPS 机器事实 + 出处注）。

C3 层划分与模型口径统一说明：模型统一为 **Qwen/Qwen3-0.6B**（28 层，`num_hidden_layers=28`，VeriEdge-repro/HF 缓存/EXO-8bit 三份 config 一致）；全部实验经 `accountedge_e2/qwen.py:26-33` 用完整 28 层划分 **C1=0–7 / C2=8–15 / C3=16–27**（C3 为 post-layer-27、pre-final-norm 边界，formal_capture `tensor_point` 实锤）捕获 C1/C2/C3 边界张量——**层划分与模型口径在所有实验内部一致**。主稿需：① 把 C3"16–23"（legacy 24 层路径遗留，见 §3.3）改为"16–27"；② 显式点名 Qwen3-0.6B（含 snapshot `c1899de2`/weights SHA `f47f7117`）；③ 区分 EXO 真实动态分片（**各 run 不同**：E3-T 本次 0–1/1–4/4–28，历史 3-node memo 1–25/25–28 等，重跑从 `/state` 实测）与受控 C1/C2/C3 checkpoint，勿混写。

---

2. B1 | GPU 口径核实

2.1 结论：`used_outside_A-F`

实证链（按强度排序）：1. **RTX6000 是真实独立机器**：`ssh RTX6000 "nvidia-smi -L"` → `NVIDIA RTX 6000D` ×2，hostname `[已脱敏]`。 2. **RTX6000 上存在捕获脚本且 mtime 精确匹配捕获时间**：`workspace/inversion/scripts/capture_720_stack.py` mtime = **2026-08-08 16:00:04**（本地 720 npz mtime = 2026-08-08 16:00–16:01）。 3. **RTX6000 有 stack_b_720 捕获副本**（`inversion/captures_720/stack_b_720/captures/`，与本地 `workspace/captures_720/stack_b_720/` **逐字节一致**，md5 相同）——捕获后同步。 4. **RTX3090 零证据**：无 capture 脚本、无 captures_720 npz、bash_history 无 capture/720 痕迹 → 排除 RTX3090 作为 stack_b 生产者。 5. **720 memo + capture 脚本 docstring** 明确写 "stack_b = RTX6000 CUDA fp32"。 6. **stack_a = mini1 实证确认**：`ssh mini1` → hostname `[已脱敏]`，持有 `inversion/captures_720/stack_a_720/captures/` 的 **720 个 npz 主副本**（mtime **2026-08-08 16:00:25**），md5 与本地一致（5819b2…），且 mini1 **无 stack_b** → mini1 是 MPS bf16 侧生产者。 7. **RTX6000 上的 stack_a 副本是镜像**：RTX6000 `inversion/captures_720/` 同时含 stack_a/stack_b，两者 md5 均与本地一致——捕获后两栈一起同步到 RTX6000 做分析；stack_a 真正产自 mini1。

与主稿冲突：主稿 `tab:eval-execution-stacks`（main_ndss.tex:948–1017）**全部 CUDA stage 用 RTX3090**，无 RTX6000。环境 inventory（`env/inventory_2026-08-06.yaml`）也只登记 mini1/mac_candidate/rtx3090，**无 RTX6000**。→ 720 池 P0/P1/P2/W8A16 的 CUDA 侧跑在 A–F 之外的真机上。

附加口径问题：– **720 "A/B" ≠ 主稿 A/B**：720 池 `capture_720_stack.py` 是一台设备**跑全 28 层**取 C1/C2/C3（同构全前缀）；主稿 A–F 是**跨设备混合 stage**（`run_full28_capture.py` 按 host 绑定，RTX3090 只做 C3+后缀）。两者是不同 stack 语义，只是重名。

上报要求（红线12）：720 池确用了 A–F 之外的设备（RTX6000）。写作侧需决定：主稿 stack 表是否加入 RTX6000 路径、或把 720 池 CUDA 侧改名为独立 stack（如 `CUDA-fp32-full`）。**不许静默用新设备出数。**

2.2 设备切换（RTX3090→RTX6000）是否影响结论——评估

切换原因：720 池搭建时，**实验室只有 1 台空闲可用的 Mac mini**（即 mini1，承担 MPS 侧 stack_a），而按主稿 A–F 定义 CUDA 侧应为 **RTX3090——但捕获期间 RTX3090 正在跑其他实验**，不可用；因此 CUDA 侧改用当时空闲的 **RTX6000** 执行 stack_b。**判定：不改变实验结论。**

理由： 1. **主导漂移分量未变**。A/B 对（`tab:eval-execution-stacks` 语义下）的 honest drift 主要来自 **bf16 vs fp32**（stack_a=MPS bf16，stack_b=CUDA fp32）：bf16 约 3 位有效数字、fp32 约 7 位，这是主导；RTX3090/RTX6000 同为 CUDA fp32，GPU 型号间的 fp32 差异是次要分量。 2. **TSTC 阈值是校准自适应的**。检测规则为 `score > Γ`，Γ 由**同一池子自己的 honest 分布**在 Q_0.99 校准而来。换设备只使 honest 分布略有移动、Γ 随之自适应；"在有界 honest drift 下能检测篡改"这一结论不变——这正是论文 tolerance-aware 的核心性质，与具体 fp32 GPU 型号无关。

仍需处理的两点（结论不变 ≠ 表格不用改）： 1. **`tab:eval-execution-stacks` 必须写真实设备**。即使结论稳健，表格写 RTX3090、数字由 RTX6000 产出，审稿人做可复现核对即挂。修法：表格加入 RTX6000 全前缀路径（或改名 `CUDA-fp32-full`）。一行改动，无理由不做。 2. **Blackwell fp32 语义脚注**。RTX6000D 为 Blackwell（sm_120），部分 Blackwell 数据中心卡的 fp32 走 fp16x2 仿真，数值行为与 RTX3090（Ampere 原生 fp32）略不同；被校准吸收，但若论文声称 stack_b 为 "FP32"，建议脚注说明架构。

> 即：**换设备可接受（正当理由 + 结论稳健），但表格如实化是论文完整性硬要求，必须做。**

---

3. B2 | C3 层划分核实

3.1 结论：`impl_16-27_is_authoritative`

实证链： 1. **模型确为 28 层**：Qwen3-0.6B `config.json` → `num_hidden_layers=28, hidden_size=1024`（本地 + HF 缓存一致）。 2. **实现划分**：`accountedge_e2/qwen.py:26-33` → C1=层 0–7、C2=层 8–15、C3=层 16–27（`layer_ids[16:28]`，覆盖全部 28 层）。qwen.py mtime **2026-08-01 21:37**，早于捕获（8/8 16:00），且 RTX6000 捕获机同一文件同一划分 → **捕获时该划分已生效，npz 的 C3 确为层 27 边界**。 3. **主稿错误**：main_ndss.tex:939 写 "C3 covers layers **16–23**" —— 对 28 层模型漏掉层 24–27（4 层）。数学上不自洽。 4. **学生手册 §1 照抄**：`NDSS27_STUDENT_EXPERIMENT_HANDBOOK(1).md:55,94` 也写 C3(层16–23)，来源即主稿。 5. **对照**：P0–E1 交付手册:129 写 "C3=层16–27后（28 层模型）" —— 与实现一致。

结论：16–27 是唯一真实划分；主稿与手册§1 的 16–23 是 stale 错误。**写作侧需改表**（主稿:939 + 手册§1）。

3.2 对 C3 层划分错误的严重性评估

判定：文档陈旧错误，非实验错误；结果零影响，但必须修，且是"论文与实现漂移"的信号。

1. **结果零影响**：没有任何实验使用 16–23 划分（全部经 qwen.py 16–27 跑出），所有数字不受影响。
2. **性质**：16–23 是 8+8+8=24 层的划分（疑似早期 24 层模型/旧 checkpoint 定义时代遗留），模型换成 28 层 Qwen3-0.6B 后主稿 methodology 段未跟着更新。
3. **风险层级**：
4. 低：不影响任何结果数值；
5. 中：主稿:939 的 C3 定义与 28 层模型不自洽（漏层 24–27），审稿人拿 config 核对可发现；
6. 高（信号）：**评估段是脱离实现独立写的**——说明 methodology 段存在与代码漂移的流程问题，不只这一处。
7. **修法**：① 改主稿:939 与手册§1 为 "C3 covers layers 16–27"（一行）；② 对主稿 eval-methodology 段做一次**对着实现重读**的审计（本 memo 即该审计的硬件/层部分）。

> 即：**分层错误是轻伤，但它是"表里不一"的系统性信号，按红线12 修表 + 审计 methodology 段。**

3.3 根因溯源（2026-08-09 补充）：legacy 24 层 vs complete 28 层路径

"16–23" 不是 typo，是历史路径语义差。 `P0_E2_EXPERIMENT_DESIGN.md:250-267`（"Full-model completeness gate"）：

- **legacy 旧路径**：受控捕获只执行层 0–23（C1=0–7, C2=8–15, C3=16–23 + final norm），**层 24–27 未执行**（等价 24 层执行）。记录标记 `controlled-legacy`，**不得支撑新 claim，与 complete 路径 "never pooled"**。
- **complete 新路径**（现行 qwen.py：C3=16–27 + final norm + LM head）：formal_capture manifest 实锤 `tensor_point=post_layer_output_pre_final_norm`（层 27 后、final norm 前），`evidence_class=controlled-input`。
- **主稿:939 / 手册§1 的 "16–23" 即 legacy 值遗留**：项目自身 Understanding.md:692 也留有未决项 "28 层 vs 24 层模型的影响? "。
- **第三个"数字不一样"来源**：EXO 真实分片是动态 place（3-node 运行切 rank0=层0–1 / rank1=层1–25 / rank2=层25–28，边界 B1≈层1、B2≈层25），与受控 harness 固定 C1/C2/C3（层7/15/27）**不是同一套划分**。论文若混写 EXO 分片边界与 C1/C2/C3，会产生第三种数字。**层数本身全模型副本恒为 28**（VeriEdge-repro/HF 缓存/EXO 8bit 三份 config 一致）。

审计结论：所有正式数字来自 complete 28 层路径；"16–23" 仅存在于论文/手册 prose。改主稿:939 + 手册§1 → "16-–27"，并在评估段写明 C3 为 post-layer-27-pre-final-norm 边界、与 EXO 动态分片区分。

---

4. B3｜模型口径核实

4.1 结论：`consistent_Qwen3-0.6B`

实证链： 1. **全部实验脚本同模型**：`grep Qwen/Qwen3-0.6B` 命中 capture_720_stack.py、determinism_cuda_100x.py、全部 ER 脚本、tokenize_720.py、SRR p1b/p4b（`workspace/models/Qwen3-0.6B`）、w8a16_redo.py。**内部一致，无第二种模型。** 2. **主稿未点名模型**：`grep -c "Qwen" main_ndss.tex` = **0**。主稿只在 line 1365 写 "one model family"，未在任何表（`tab:pair-family-projcos`、`tab:eval-execution-stacks` 等）声明模型名。 3. **交叉核对**：`tab:pair-family-projcos` 等表数字来源脚本（run_fixed_fpr_comparison.py、run_p0_main_table*.py）→ 同 Qwen3-0.6B。

结论：实验内部一致；主稿需显式点名 Qwen3-0.6B（并给 snapshot `c1899de2` / weights SHA `f47f7117`，P0-E1 手册:128 已有）。

---

5. 异常与坑

1. **RTX6000 的 captures_720 是镜像而非主副本**：与本地 md5 逐字节相同（5819b229…/83dd9f…）。说明捕获后做了一次同步；不影响"RTX6000 是 CUDA 侧生产者"的结论（脚本+mtime+RTX3090 零证据已锁），但**没有捕获时执行日志**，GPU 归属靠"脚本存在 + mtime + memo"三角而非进程日志。
2. ~~stack_a 的 MPS 机器无法回溯~~ —— **已解决（2026-08-09 复核）**：mini1 持有 stack_a 主副本（720 npz、mtime 16:00:25、md5 与本地一致、无 stack_b），确认 stack_a = mini1（§2.1 证据 6–7）。
3. **"A/B" 命名与主稿 A/B 冲突**：同构全前缀 vs 混合 stage，两套语义。
4. **eval FPR 对 q 敏感**（A1.3 连带发现）：q=64/seed=2026 → 0.96%，q=256 → 1.92%。与层划分无关但影响阈值口径稳定性。

---

6. 一句话结论

`used_outside_A-F` + `impl_16-27_is_authoritative` + `consistent_Qwen3-0.6B`：720 池 CUDA 侧确在 RTX6000（RTX6000D）上捕获、不在主稿 A–F（RTX3090）内（红线12 上报）；C3 真实划分是 16–27（主稿/手册 16–23 错误，需改表）；模型全实验统一 Qwen3-0.6B（主稿需点名）。

7. 本次没做的部分

- 捕获进程级执行日志（RTX6000 无留存，不可回溯）——GPU 归属靠脚本+mtime+memo 三角
- QwenFull28Adapter 重跑逐张量复现 npz（可选；qwen.py mtime<捕获时间 + config 28 层已足证边界）
- 主稿 stack 表/层划分/模型名的具体改法（写作侧裁决）